

Lab Report No 7
Implementation of Edge Detection
Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

15th Nov, 2024

Department of Computer Engineering,
HITEC University, Taxila

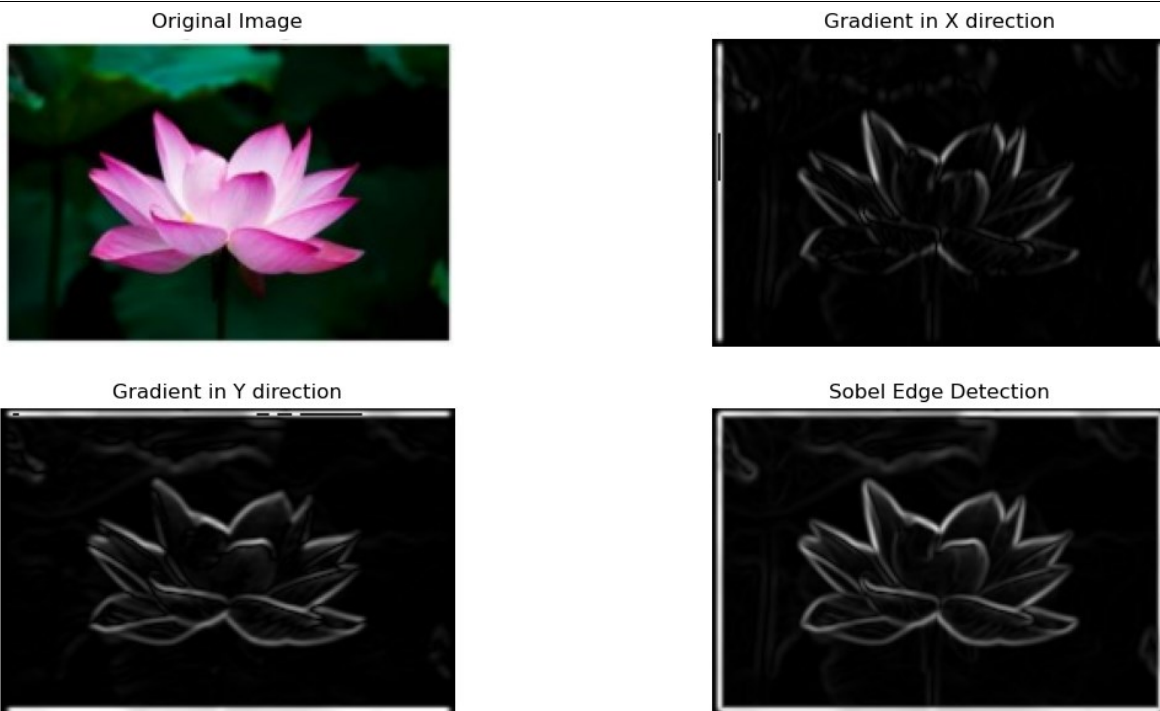
Lab Example No 1:

Solution:

Brief description (3-5 lines)
Sobel filter: Sobel filter calculates the first order derivative of the image. Horizontal Sobel filter is used to find out the edges along x-axis as it calculates the derivative of an image in x direction while Vertical Sobel filter is used to find out the edges along y-axis. The edges obtained from applying both this filters can then be added to obtain all the edges in an image.
The code
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt # Load the image image_path = 'flower.jpg' # Replace with your image path image = cv2.imread(image_path) # Convert to grayscale gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # Apply Gaussian smoothing (optional) blurred_image = cv2.GaussianBlur(gray_image, (3, 3), 0) # Sobel operators Gx = cv2.Sobel(blurred_image, cv2.CV_64F, 1, 0, ksize=3) Gy = cv2.Sobel(blurred_image, cv2.CV_64F, 0, 1, ksize=3) # Gradient magnitude G = np.sqrt(Gx**2 + Gy**2) # Normalize to range 0-255 Gx = np.uint8(255 * np.abs(Gx) / np.max(Gx)) Gy = np.uint8(255 * np.abs(Gy) / np.max(Gy)) G = np.uint8(255 * G / np.max(G)) # Display the results plt.figure(figsize=(15, 10)) # Original image plt.subplot(2, 2, 1) plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))</pre>

```
plt.title('Original Image')
plt.axis('off')
# Gradient in X direction
plt.subplot(2, 2, 2)
plt.imshow(Gx, cmap='gray')
plt.title('Gradient in X direction')
plt.axis('off')
# Gradient in Y direction
plt.subplot(2, 2, 3)
plt.imshow(Gy, cmap='gray')
plt.title('Gradient in Y direction')
plt.axis('off')
# Edge-detected image
plt.subplot(2, 2, 4)
plt.imshow(G, cmap='gray')
plt.title('Sobel Edge Detection')
plt.axis('off')
plt.show()
```

The results (Screenshot)



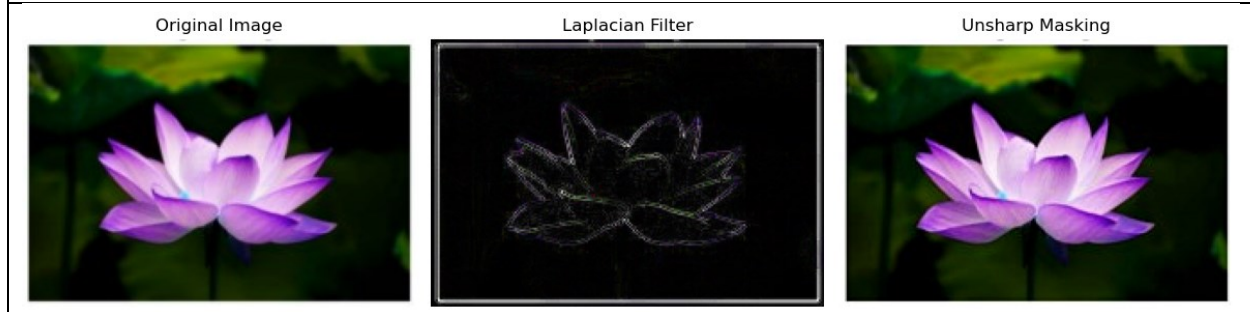
Lab Example No 2:

Solution:

Brief description (3-5 lines)
Laplacian Filter: Calculates the second derivative of the image to emphasize edges. Unsharp Masking: A technique where a blurred version of the image is subtracted from the original.
The code
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt # Load the image image_path = 'flower.jpg' # Replace with your image path image = cv2.imread(image_path) # Apply Laplacian Filter laplacian_filter = cv2.Laplacian(image, cv2.CV_64F) laplacian_filter = cv2.convertScaleAbs(laplacian_filter) # Convert to 8-bit # Unsharp Masking gaussian_blur = cv2.GaussianBlur(image, (5, 5), sigmaX=1) unsharp_image = cv2.addWeighted(image, 1.5, gaussian_blur, -0.5, 0) # Display Results plt.figure(figsize=(12, 5)) plt.subplot(1, 3, 1) plt.imshow(image, cmap='gray') plt.title('Original Image') plt.axis('off') plt.subplot(1, 3, 2) plt.imshow(laplacian_filter, cmap='gray') plt.title('Laplacian Filter') plt.axis('off') plt.subplot(1, 3, 3)</pre>

```
plt.imshow(unsharp_image, cmap='gray')
plt.title('Unsharp Masking')
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Example No 3(a):

Solution:

Brief description (3-5 lines)

Canny Edge Detection:

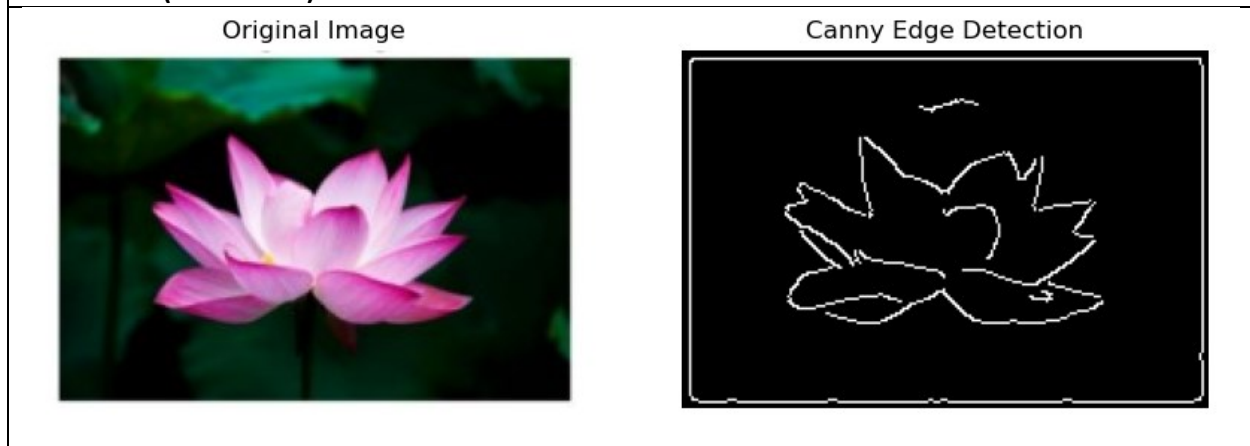
Canny Algorithm is known for its optimal edge detection capabilities. The algorithm follows a series of steps to reduce noise, detect edges, and improve the accuracy of edge detection.

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image
image_path = 'flower.jpg' # Replace with your image path
image = cv2.imread(image_path, cv2.IMREAD_COLOR)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Gaussian blur to reduce noise
blurred_image = cv2.GaussianBlur(gray_image, (5, 5), 1.4)
# Apply Canny edge detector
edges = cv2.Canny(blurred_image, 100, 200)
# Display the result
plt.figure(figsize=(10, 5))
```

```
# Original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
plt.axis('off')
# Edge-detected image
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Detection')
plt.axis('off')
plt.show()
```

The results (Screenshot)



Lab Task#1:

Solution:

Brief description (3-5 lines)

Apply different edge detection techniques on the above given image and plot result.

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image
image = cv2.imread('flower.jpg', cv2.IMREAD_GRAYSCALE)
# Sobel Edge Detection
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
```

```

sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)
# Canny Edge Detection
canny = cv2.Canny(image, 100, 200)
# Laplacian Edge Detection
laplacian = cv2.Laplacian(image, cv2.CV_64F)
# Prewitt Edge Detection
kernelx = np.array([[1, 0, -1], [1, 0, -1], [1, 0, -1]])
kernely = np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]])
prewitt_x = cv2.filter2D(image, -1, kernelx)
prewitt_y = cv2.filter2D(image, -1, kernely)
prewitt = cv2.magnitude(prewitt_x.astype(np.float64),
prewitt_y.astype(np.float64))
# Display the results
plt.figure(figsize=(12, 8))
plt.subplot(231), plt.imshow(image, cmap='gray'), plt.title('Original
Image')
plt.axis('off')
plt.subplot(232), plt.imshow(sobel, cmap='gray'), plt.title('Sobel Edge
Detection')
plt.axis('off')
plt.subplot(233), plt.imshow(canny, cmap='gray'), plt.title('Canny
Edge Detection')
plt.axis('off')
plt.subplot(234), plt.imshow(laplacian, cmap='gray'),
plt.title('Laplacian Edge Detection')
plt.axis('off')
plt.subplot(235), plt.imshow(prewitt, cmap='gray'), plt.title('Prewitt
Edge Detection')
plt.axis('off')
plt.tight_layout()
plt.show()

```

The results (Screenshot)

